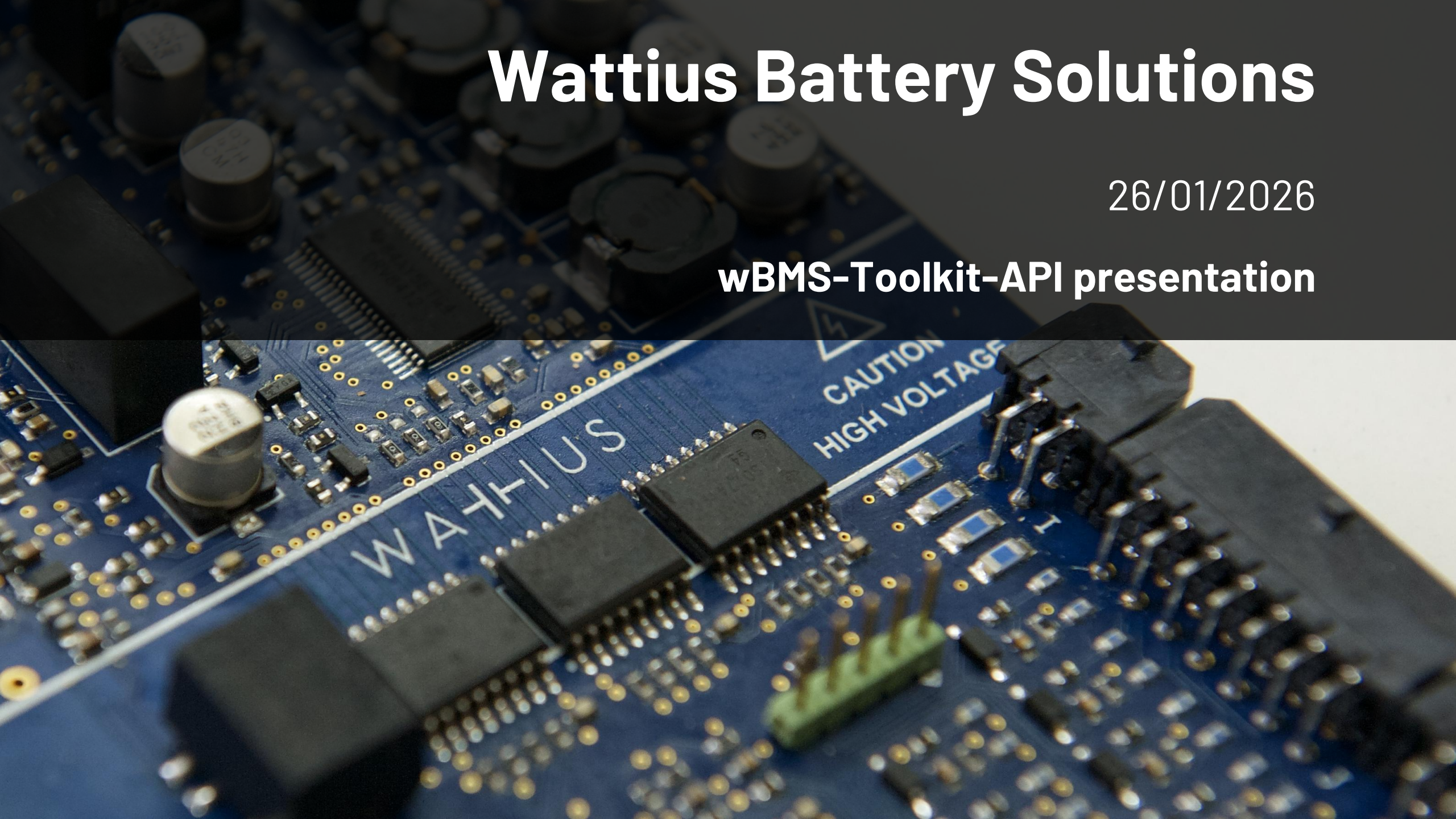


Wattius Battery Solutions

26/01/2026

wBMS-Toolkit-API presentation

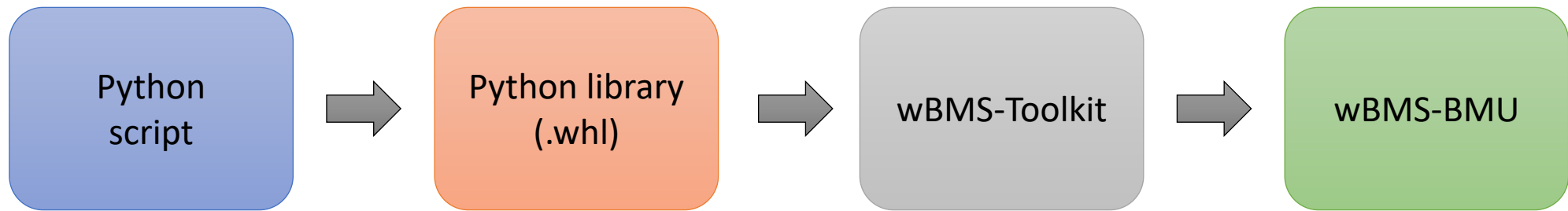


Index

- 1. Introduction**
- 2. Installation**
- 3. End of Line script**

1. Introduction

Python library for automatic manipulation of the wBMS-Toolkit.



1. Introduction

Functionalities from the Python library (.whl)

- Modify the BMU Setup parameters
- Upload the configuration files
- Update the firmware(s)
- Configure the connection settings between the wBMS-Toolkit and the wBMS-BMU.
- Force the wBMS-BMU to a specific SoC
- Get configuration CRC from wBMS-BMU
- Log in to wBMS-Toolkit
- Start and stop a log trace.
- Get the assigned IP of the wBMS-BMU

2. Installation

Installation requirements

- Python 3
- Install the .whl file from the wBMS-Toolkit-API:
 - Download the .whl file from de Wattius Services repository (for example: *wbms_api-1.0.0-py3-none-any.whl*)
 - Execute the command "*pip install path/to/file.whl*".
 - Import the library into the Python scripts.

```
from wbms_api.wbms_toolkit_api import *  
from wbms_api.wbms_toolkit_types import *
```

- Install external Python libraries: *socket*, *select* y *time*

3. End of Line script

Example script to perform the End-of-Line process

- Update firmware(s)
- Upload configurations

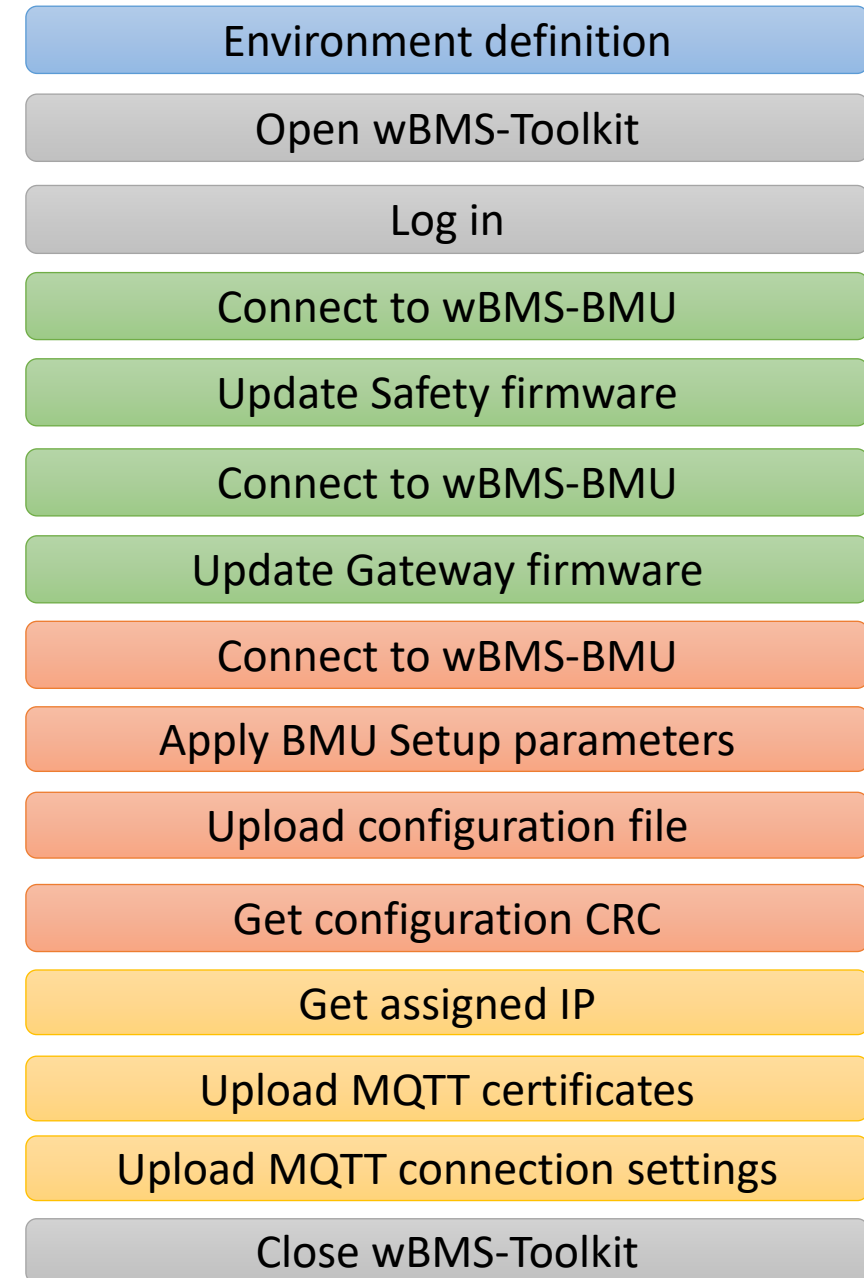
Installation requirements:

- Same installation requirements as the Python library.
- Install external Python libraries: *subprocess, sys, os, urllib.parse y requests*

3. End of Line script

Process

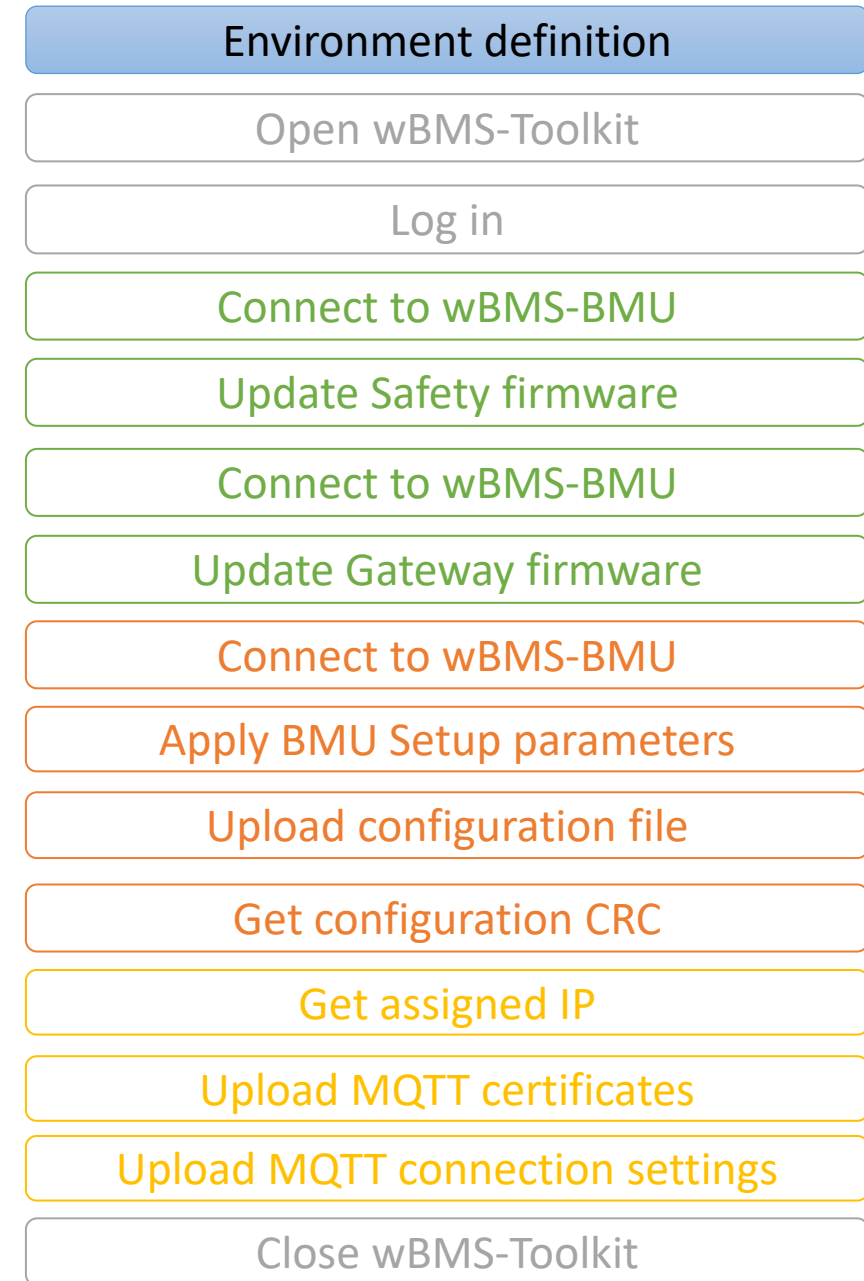
1. Environment definition
2. wBMS-Toolkit initialization
3. Update firmware(s)
4. Upload configuration
5. Upload MQTT connectivity settings
6. Close wBMS-Toolkit



3. End of Line script

1. Environment definition

- Part of the code in charge of getting all the required information to execute the rest of steps. For example:
 - Path to the wBMS-Toolkit file
 - Connection mode between wBMS-Toolkit and wBMS-BMU
- In the example script, this information is requested by console. Nevertheless, it can be modified and defined directly on the code.



3. End of Line script

2. wBMS-Toolkit initialization

- Part of the code in charge of opening the wBMS-Toolkit and logging in using the defined credentials.
- Between the "Open wBMS-Toolkit" and "Log in" commands, the "wbms_toolkit_api_poll" function is called to wait until the wBMS-Toolkit-API is available.
 - The maximum waiting time can be defined when this function is called (10 seconds by default).
- In case that a user is already logged in into the wBMS-Toolkit, the "Log in" step can be ignored.
- The current implementation requires Internet connection every time that this step is executed. To avoid having Internet connection every time, open the wBMS-Toolkit manually and select the "Keep me logged in" checkbox during the log in. Then, every time that the wBMS-Toolkit is opened automatically, the user credentials aren't required.

Environment definition

Open wBMS-Toolkit

Log in

Connect to wBMS-BMU

Update Safety firmware

Connect to wBMS-BMU

Update Gateway firmware

Connect to wBMS-BMU

Apply BMU Setup parameters

Upload configuration file

Get configuration CRC

Get assigned IP

Upload MQTT certificates

Upload MQTT connection settings

Close wBMS-Toolkit

3. End of Line script

2. wBMS-Toolkit initialization

- Check the next points when the API returns error in this step:
 - Incorrect credentials
 - No Internet connection

Environment definition

Open wBMS-Toolkit

Log in

Connect to wBMS-BMU

Update Safety firmware

Connect to wBMS-BMU

Update Gateway firmware

Connect to wBMS-BMU

Apply BMU Setup parameters

Upload configuration file

Get configuration CRC

Get assigned IP

Upload MQTT certificates

Upload MQTT connection settings

Close wBMS-Toolkit

3. End of Line script

2. wBMS-Toolkit initialization

```
## Start process
print("***** OPENING TOOLKIT *****")
proc = subprocess.Popen([toolkit_path])

try:
    if (wbms_toolkit_api_poll() == False):
        proc.terminate()
        try:
            proc.wait(timeout=5)
        except subprocess.TimeoutExpired:
            proc.kill()
        sys.exit()

    ## Sign in
    print("***** SIGN IN *****")
    log_in_result = wbms_sign_in(toolkit_credential_user, toolkit_credential_password)
    if (log_in_result != wbms_result.OK):
        proc.terminate()
        try:
            proc.wait(timeout=5)
        except subprocess.TimeoutExpired:
            proc.kill()
        sys.exit()
```

Environment definition

Open wBMS-Toolkit

Log in

Connect to wBMS-BMU

Update Safety firmware

Connect to wBMS-BMU

Update Gateway firmware

Connect to wBMS-BMU

Apply BMU Setup parameters

Upload configuration file

Get configuration CRC

Get assigned IP

Upload MQTT certificates

Upload MQTT connection settings

Close wBMS-Toolkit

3. End of Line script

3. Update firmware(s)

- Part of the code in charge of updating the firmware(s):
 - Safety
 - Gateway
- If the wBMS-BMU is in bootloader mode, the first "Connect to wBMS-BMU" step shall return ERROR.
- The update process has a timeout of 6 minutes.

Environment definition

Open wBMS-Toolkit

Log in

Connect to wBMS-BMU

Update Safety firmware

Connect to wBMS-BMU

Update Gateway firmware

Connect to wBMS-BMU

Apply BMU Setup parameters

Upload configuration file

Get configuration CRC

Get assigned IP

Upload MQTT certificates

Upload MQTT connection settings

Close wBMS-Toolkit

3. End of Line script

3. Update firmware(s)

- Check the next points when the API returns error in this step:
 - Incorrect firmware(s)
 - The wBMS-BMU is not connected to wBMS-Toolkit
 - The connection settings from the wBMS-Toolkit are not the same that the physical connection with the wBMS-BMU.

Environment definition

Open wBMS-Toolkit

Log in

Connect to wBMS-BMU

Update Safety firmware

Connect to wBMS-BMU

Update Gateway firmware

Connect to wBMS-BMU

Apply BMU Setup parameters

Upload configuration file

Get configuration CRC

Get assigned IP

Upload MQTT certificates

Upload MQTT connection settings

Close wBMS-Toolkit

3. End of Line script

3. Update firmware(s)

```
## Connect to BMU
print("***** CONNECTING TO BMU *****")
if (connection_mode == "USB"):
    print(wbms_set_connection_usb())
elif (connection_mode == "CAN"):
    print(wbms_set_connection_can(can_convert_type, can_baudrate, can_node_id, wbms_can_toolkit_refresh.FAST))

## Start Updating firmware
print("***** UPDATING SAFETY FIRMWARE *****")
print(wbms_update_firmware(safety_firmware_path))

if(gateway_firmware_path != ""):
    ## Connect to BMU
    print("***** CONNECTING TO BMU *****")
    if (connection_mode == "USB"):
        print(wbms_set_connection_usb())
    elif (connection_mode == "CAN"):
        print(wbms_set_connection_can(can_convert_type, can_baudrate, can_node_id, wbms_can_toolkit_refresh.FAST))

    ## Start Updating firmware
    print("***** UPDATING GATEWAY FIRMWARE *****")
    print(wbms_update_firmware(gateway_firmware_path))
    time.sleep(5)
```

Environment definition

Open wBMS-Toolkit

Log in

Connect to wBMS-BMU

Update Safety firmware

Connect to wBMS-BMU

Update Gateway firmware

Connect to wBMS-BMU

Apply BMU Setup parameters

Upload configuration file

Get configuration CRC

Get assigned IP

Upload MQTT certificates

Upload MQTT connection settings

Close wBMS-Toolkit

3. End of Line script

4. Upload configuration

- Part of the code in charge of uploading the configuration parameters to wBMS-BMU
- When requesting to upload configuration parameters, the wBMS-Toolkit-API can return:
 - "WAITING": No wBMS-BMU is connected to the wBMS-Toolkit.
 - "GETTING": The wBMS-Toolkit is getting the configuration from the wBMS-BMU. So, it cannot upload the new requested configuration in that moment.
- The timeout of the upload configuration process is 90 seconds.

Environment definition

Open wBMS-Toolkit

Log in

Connect to wBMS-BMU

Update Safety firmware

Connect to wBMS-BMU

Update Gateway firmware

Connect to wBMS-BMU

Apply BMU Setup parameters

Upload configuration file

Get configuration CRC

Get assigned IP

Upload MQTT certificates

Upload MQTT connection settings

Close wBMS-Toolkit

3. End of Line script

4. Upload configuration

- Check the next points when the API returns error in this step :
 - Incorrect configuration file
 - Incorrect BMU Setup parameters
 - The wBMS-BMU is not connected to wBMS-Toolkit
 - The connection settings from the wBMS-Toolkit are not the same that the physical connection with the wBMS-BMU.

Environment definition

Open wBMS-Toolkit

Log in

Connect to wBMS-BMU

Update Safety firmware

Connect to wBMS-BMU

Update Gateway firmware

Connect to wBMS-BMU

Apply BMU Setup parameters

Upload configuration file

Get configuration CRC

Get assigned IP

Upload MQTT certificates

Upload MQTT connection settings

Close wBMS-Toolkit

3. End of Line script

4. Upload configuration

```
## Connect to BMU
print("***** CONNECTING TO BMU *****")
if (connection_mode == "USB"):
    print(wbms_set_connection_usb())
elif (connection_mode == "CAN"):
    print(wbms_set_connection(can_convert_type, can_baudrate, can_node_id, wbms_can_toolkit_refresh.FAST))

## Apply BMU setup
print("***** APPLYING BMU SETUP *****")
apply_bmu_setup(setup_can_baudrate, setup_can_node_id, setup_bluetooth_mode, setup_network_mode, setup_static_ip_address,

## Upload configuration
print("***** UPLOADING CONFIGURATION *****")
upload_configuration(config_path)
time.sleep(10)

## Get configuration CRC
print("***** GETTING CONFIGURATION CRC *****")
result = wbms_get_config_crc()
timeout = 30
start = time.monotonic()
while (result == wbms_result.WAITING):
    if time.monotonic() - start > timeout:
        raise TimeoutError("Config CRC could not be read")
    result = wbms_get_config_crc()
if(result == "FAILURE"):
    print(result)
else:
    print("Configuration CRC (dec):" + str(int(result)))
    print("Configuration CRC (hex):" + str(hex(int(result))))
```

Environment definition

Open wBMS-Toolkit

Log in

Connect to wBMS-BMU

Update Safety firmware

Connect to wBMS-BMU

Update Gateway firmware

Connect to wBMS-BMU

Apply BMU Setup parameters

Upload configuration file

Get configuration CRC

Get assigned IP

Upload MQTT certificates

Upload MQTT connection settings

Close wBMS-Toolkit

3. End of Line script

4. Upload configuration

```
def upload_configuration(config_path):
    result = wbms_result.WAITING
    timeout = 90
    start = time.monotonic()
    while ((result != wbms_result.ERROR) & (result != wbms_result.OK) & (result != wbms_result.TIMEOUT)):
        if time.monotonic() - start > timeout:
            raise TimeoutError("Configuration could not uploaded")
        result = wbms_upload_config(config_path)
    print(result)

def apply_bmu_setup(setup_can_baudrate, setup_can_node_id, setup_bluetooth_mode, setup_network_mode, setup_static_ip_address, setup_static_ip_mask):
    result = wbms_result.WAITING
    timeout = 60
    start = time.monotonic()
    while ((result != wbms_result.ERROR) & (result != wbms_result.OK) & (result != wbms_result.TIMEOUT)):
        if time.monotonic() - start > timeout:
            raise TimeoutError("BMU Setup could not applied")
        result = wbms_apply_setup(setup_can_baudrate, setup_can_node_id, setup_bluetooth_mode, setup_network_mode, setup_static_ip_address, setup_static_ip_mask)
    print(result)
```

Environment definition

Open wBMS-Toolkit

Log in

Connect to wBMS-BMU

Update Safety firmware

Connect to wBMS-BMU

Update Gateway firmware

Connect to wBMS-BMU

Apply BMU Setup parameters

Upload configuration file

Get configuration CRC

Get assigned IP

Upload MQTT certificates

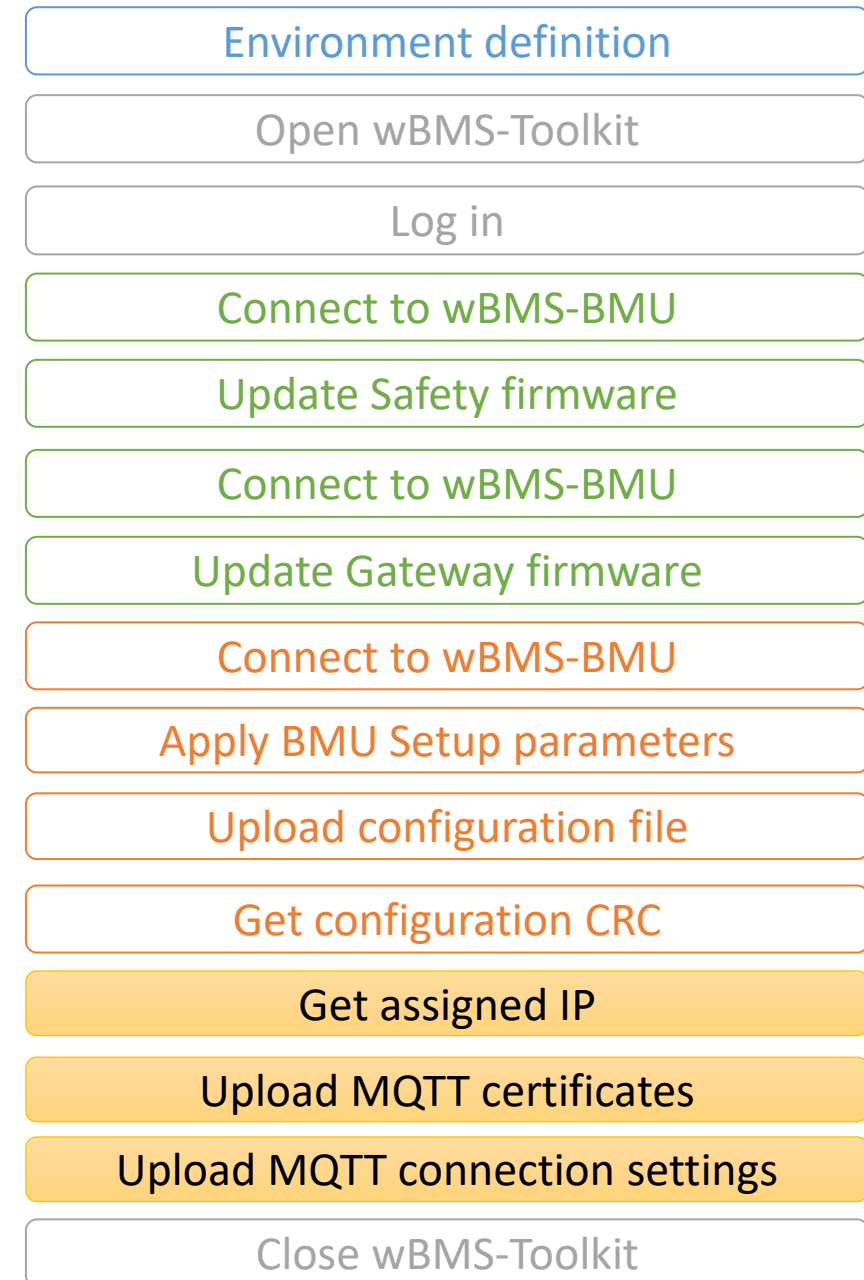
Upload MQTT connection settings

Close wBMS-Toolkit

3. End of Line script

5. Upload MQTT connectivity settings

- Part of the code in charge of uploading the MQTT connectivity settings to wBMS-BMU
- If the MQTT connectivity is not used, this step can be ignored.
- Since the "Network mode" is configured, it can take up to 20 seconds to receive the assigned IP.
- When no IP is assigned, the wBMS-Toolkit-API returns "0.0.0.0"



3. End of Line script

5. Upload MQTT connectivity settings

- Check the next points when the returned IP is not a valid IP:
 - When the wBMS-Toolkit-API returns "0.0.0.0" means that the wBMS-BMU doesn't have Ethernet connection.
 - When the wBMS-Toolkit-API returns "OK" means that the received "Network mode" from the wBMS-BMU is "Disable".
 - When the wBMS-Toolkit-API returns "WAITING" means that the wBMS-Toolkit has not received the assigned IP from the wBMS-BMU yet.

Environment definition

Open wBMS-Toolkit

Log in

Connect to wBMS-BMU

Update Safety firmware

Connect to wBMS-BMU

Update Gateway firmware

Connect to wBMS-BMU

Apply BMU Setup parameters

Upload configuration file

Get configuration CRC

Get assigned IP

Upload MQTT certificates

Upload MQTT connection settings

Close wBMS-Toolkit

3. End of Line script

5. Upload MQTT connectivity settings

```
## Gateway configuration
if (upload_mqtt_certificates == "Y"):
    ## Get IP
    print("***** GETTING IP *****")
    timeout = 30
    start = time.monotonic()
    result = wbms_result.OK
    ip_address = "0.0.0.0"
    while ((result == wbms_result.WAITING) | (result == wbms_result.OK) | (ip_address == "0.0.0.0")):
        if time.monotonic() - start > timeout:
            raise TimeoutError("IP don't get in time")
        result = wbms_get_ip()
        if ((result != wbms_result.WAITING) & (result != wbms_result.OK)):
            ip_address_parts = result.decode("utf-8").split('.')
            ip_address = str(int(ip_address_parts[0])) + "." + str(int(ip_address_parts[1])) + "." + str(int(ip_address_parts[2])) + "." + str(int(ip_address_parts[3]))

    print("IP: " + ip_address)
    url = f"http://{ip_address}"

    ## Upload MQTT certificates
    print("***** UPLOADING MQTT CERTIFICATES *****")
    with open(pem_path, "rb") as f: pem_cert_txt = f.read()
    with open(crt_path, "rb") as f: crt_cert_txt = f.read()
    with open(key_path, "rb") as f: key_client_txt = f.read()

    pem_cert = pem_cert_txt.decode("utf-8")
    crt_cert = crt_cert_txt.decode("utf-8")
    key_client = key_client_txt.decode("utf-8")

    pem = urllib.parse.quote(pem_cert, safe=";/?:@&+$_,-.~*'()*#")
    crt = urllib.parse.quote(crt_cert, safe=";/?:@&+$_,-.~*'()*#")
    key = urllib.parse.quote(key_client, safe=";/?:@&+$_,-.~*'()*#")

    mqtt_url_url = url + "/api/v1/mcu2/config/mqttBroker?content=" + mqtt_url
    pem_url = url + "/api/v1/mcu2/config/servercert?content=" + pem
    crt_url = url + "/api/v1/mcu2/config/clientcert?content=" + crt
    key_url = url + "/api/v1/mcu2/config/clientkey?content=" + key
```

Environment definition

Open wBMS-Toolkit

Log in

Connect to wBMS-BMU

Update Safety firmware

Connect to wBMS-BMU

Update Gateway firmware

Connect to wBMS-BMU

Apply BMU Setup parameters

Upload configuration file

Get configuration CRC

Get assigned IP

Upload MQTT certificates

Upload MQTT connection settings

Close wBMS-Toolkit

3. End of Line script

5. Upload MQTT connectivity settings

```
# 1) Upload Server Certificate (.pem)
try:
    r_pem = requests.get(pem_url, timeout=5)
    if r_pem.status_code == 200:
        print(f"(Server CERT (.pem))\t\t-> {r_pem.text.strip()}")
    else:
        print(f"⚠ Server CERT failed: HTTP {r_pem.status_code}")
except requests.exceptions.RequestException as e:
    print(f"❌ Error saving Server CERT: {e}")

# 2) Upload Client Certificate (.crt)
try:
    r.crt = requests.get(crt_url, timeout=5)
    if r.crt.status_code == 200:
        print(f"(Client CERT (.crt))\t\t-> {r.crt.text.strip()}")
    else:
        print(f"⚠ Client CERT failed: HTTP {r.crt.status_code}")
except requests.exceptions.RequestException as e:
    print(f"❌ Error saving Client CERT: {e}")

# 3) Upload Client Key (.key)
try:
    r_key = requests.get(key_url, timeout=5)
    if r_key.status_code == 200:
        print(f"(Client KEY (.key))\t\t-> {r_key.text.strip()}")
    else:
        print(f"⚠ Client KEY failed: HTTP {r_key.status_code}")
except requests.exceptions.RequestException as e:
    print(f"❌ Error saving Client KEY: {e}")

# 4) Save MQTT URL
try:
    r_mqtt_url = requests.get(mqtt_url_url, timeout=5)
    if r_mqtt_url.status_code == 200:
        print(f"(MQTT URL)\t\t\t-> {r_mqtt_url.text.strip()}")
    else:
        print(f"⚠ MQTT URL failed: HTTP {r_mqtt_url.status_code}")
except requests.exceptions.RequestException as e:
    print(f"❌ Error saving MQTT URL: {e}")

if (upload_mqtt_connection_settings == "Y"):
    mqtt_connection_settings_url = url + "/api/v1/mcu2/config/flags?measurements_events=" + mqtt_conn_settings_measurements_events + "&mqtt_toolkit_connection=" +
    try:
        r_mqtt_conn_settings = requests.get(mqtt_connection_settings_url, timeout=5)
        if r_mqtt_conn_settings.status_code == 200:
            print(f"(MQTT connection settings)\t\t-> {r_mqtt_conn_settings.text.strip()}")
        else:
            print(f"⚠ MQTT connection settings failed: HTTP {r_mqtt_conn_settings.status_code}")
    except requests.exceptions.RequestException as e:
        print(f"❌ Error saving MQTT connection settings: {e}")
```

Environment definition

Open wBMS-Toolkit

Log in

Connect to wBMS-BMU

Update Safety firmware

Connect to wBMS-BMU

Update Gateway firmware

Connect to wBMS-BMU

Apply BMU Setup parameters

Upload configuration file

Get configuration CRC

Get assigned IP

Upload MQTT certificates

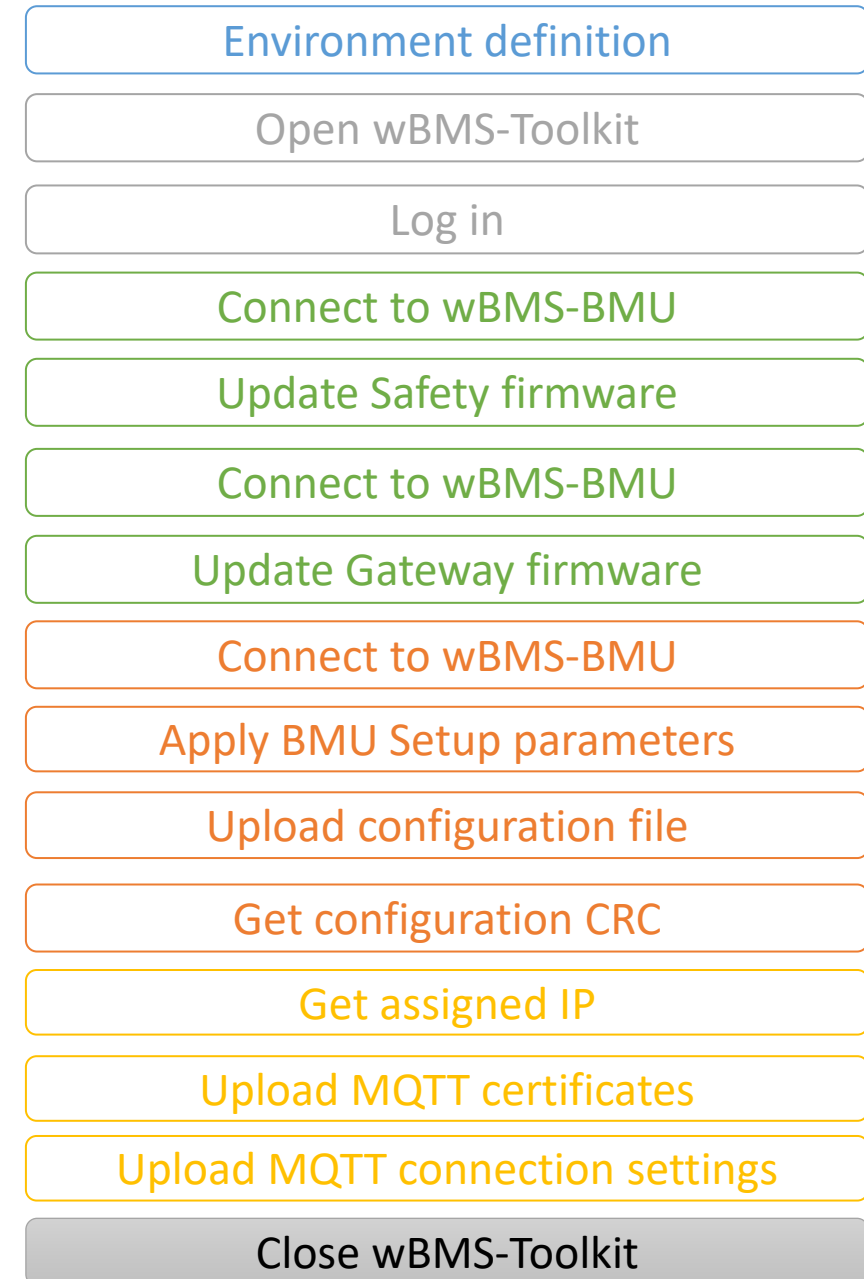
Upload MQTT connection settings

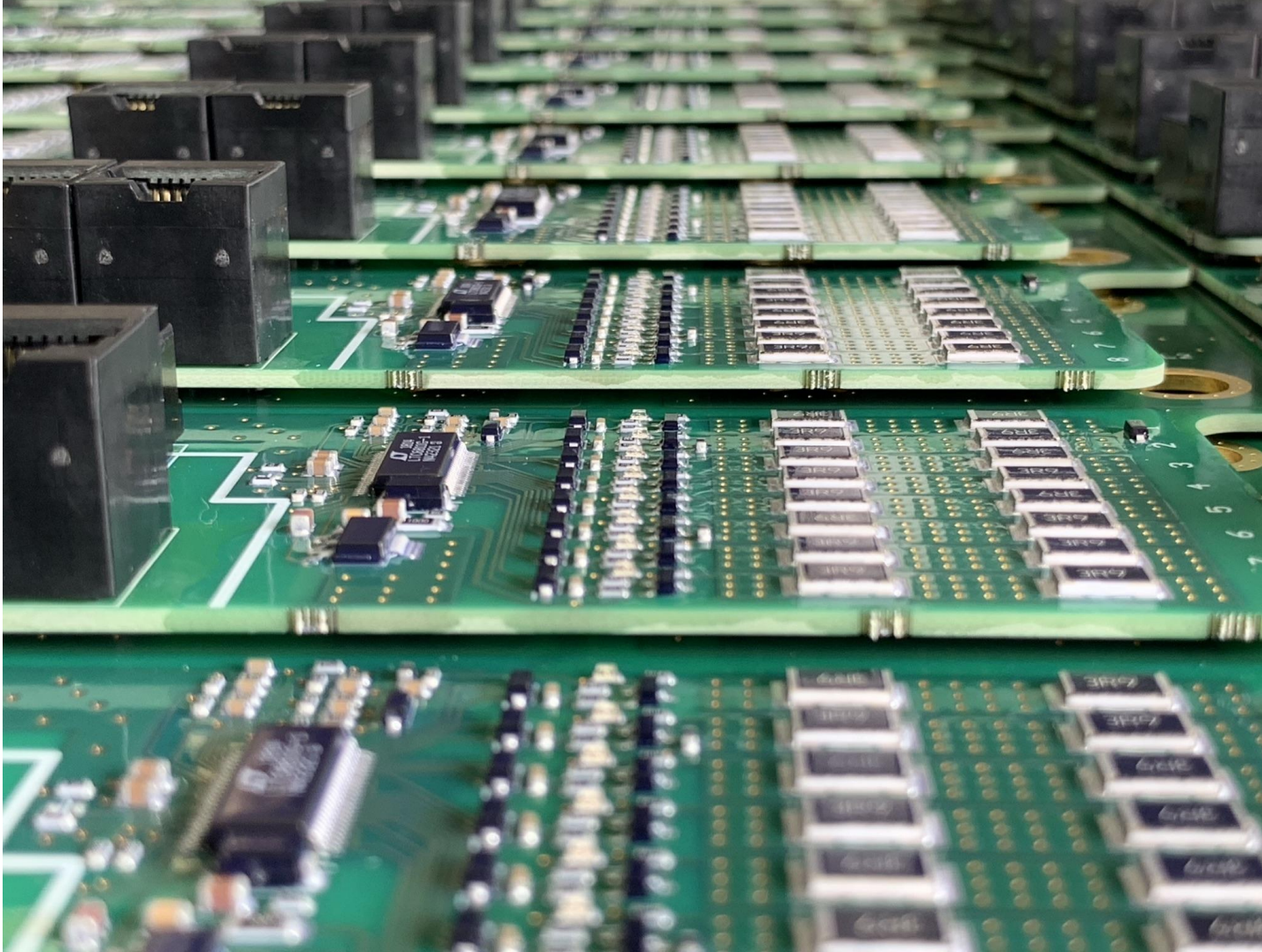
Close wBMS-Toolkit

3. End of Line script

6. Close wBMS-Toolkit

- Part of the code in charge of closing the wBMS-Toolkit.





Thank you!

Wattius Battery Solutions

Camí de Can Bassa 6
08401 Granollers
Barcelona

info@wattius.com
www.wattius.com